Data Structures and Algorithms Lab
Assignment 6
Experiment 6 (Binary Search Tree and Heap)

- To demonstrate the creation of a **binary search tree** and implement its operation.
- To create a **binary heap** data structure and implement its operations, and its applications.

Instruction: Make a pdf file of the program code in C and its output.

Name: Chaitanya Sharma
SAP ID: 590026716
Batch: 39
Link of your GitHub Account:
https://github.com/Chaitanya189

Course: B.Tech. CSE (Sem II)
Subject: DSA Lab
Date: 14 April 2026
Note: Do all the implementations in C programming.

1. Write a C program to implement a **Binary Search Tree** (BST) using a linked list representation. Your program should support the following operations:
    (i) **Insertion**: Insert a node into the BST while maintaining BST properties. Duplicate values should not be allowed.
    (ii) **Deletion**: Delete a node from the BST based on a given key.
        - Handle all three cases: (a) Node is a leaf node, (b) Node has one child and (c) Node has two children
    (iii) **Searching**: Search for a given value in the BST (Display whether the element is found or not).

(iv) **Traversal**: In order Traversal (Left → Root → Right)

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* newNode(int val) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));
    n->data = val; n->left = n->right = NULL;
    return n;
}

struct Node* insert(struct Node* root, int val) {
    if (!root) return newNode(val);
    if (val < root->data)       root->left  = insert(root->left,  val);
    else if (val > root->data)  root->right = insert(root->right, val);
    else printf("Duplicate value %d not allowed.\n", val);
    return root;
}

struct Node* minNode(struct Node* root) {
    while (root->left) root = root->left;
    return root;
}

struct Node* delete(struct Node* root, int val) {
    if (!root) { printf("Value %d not found.\n", val); return NULL; }
    if (val < root->data)
        root->left = delete(root->left, val);
    else if (val > root->data)
        root->right = delete(root->right, val);
    else {
        if (!root->left)  { struct Node* t = root->right; free(root); return
            t; }
        if (!root->right) { struct Node* t = root->left;  free(root); return
            t; }
        struct Node* succ = minNode(root->right);
        root->data = succ->data;
        root->right = delete(root->right, succ->data);
    }
    return root;
}
```

```c
void search(struct Node* root, int val) {
    if (!root)            { printf("Element %d NOT found.\n", val); return;
        }
    if (val == root->data){ printf("Element %d FOUND.\n", val); return; }
    if (val < root->data) search(root->left, val);
    else                  search(root->right, val);
}

void inorder(struct Node* root) {
    if (!root) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

int main() {
    struct Node* root = NULL;
    int choice, val;

    while (1) {
        printf("\n--- BST Menu ---\n");
        printf("1.Insert  2.Delete  3.Search  4.Inorder  5.Exit\n");
        printf("Enter choice: "); scanf("%d", &choice);

        switch (choice) {
            case 1: printf("Enter value: "); scanf("%d", &val);
                    root = insert(root, val); break;
            case 2: printf("Enter value to delete: "); scanf("%d", &val);
                    root = delete(root, val); break;
            case 3: printf("Enter value to search: "); scanf("%d", &val);
                    search(root, val); break;
            case 4: printf("Inorder: "); inorder(root); printf("\n"); break;
            case 5: return 0;
            default: printf("Invalid choice!\n");
        }
    }
}
```

OUTPUT :

```
--- BST Menu ---
1.Insert  2.Delete  3.Search  4.Inorder  5.Exit
Enter choice: 1
Enter value: 22 12 3

--- BST Menu ---
1.Insert  2.Delete  3.Search  4.Inorder  5.Exit
Enter choice: Invalid choice!

--- BST Menu ---
1.Insert  2.Delete  3.Search  4.Inorder  5.Exit
Enter choice: Enter value to search: 1
Element 1 NOT found.

--- BST Menu ---
1.Insert  2.Delete  3.Search  4.Inorder  5.Exit
Enter choice: 1
Enter value: 56

--- BST Menu ---
1.Insert  2.Delete  3.Search  4.Inorder  5.Exit
Enter choice: 3
Enter value to search: 3
Element 3 NOT found.

--- BST Menu ---
1.Insert  2.Delete  3.Search  4.Inorder  5.Exit
Enter choice: 4
Inorder: 22 56

--- BST Menu ---
1.Insert  2.Delete  3.Search  4.Inorder  5.Exit
Enter choice:
```

2. Write a C program to implement a **Binary Heap** (Min Heap or Max Heap). The program should support the following operations:
   (i) **Heap Creation**: Initialize an empty heap using an array.
   (ii) **Insertion (Heapify-Up):** Insert a new element into the heap. Maintain the heap property using heapify-up operation.
   (iii) **Deletion (Heapify-Down):** Delete the root element (minimum in Min Heap / maximum in Max Heap). Maintain heap property using heapify-down operation.
   (iv) **Display Heap:** Print the elements of the heap.

```c
#include <stdio.h>
#define MAX 100

int heap[MAX], size = 0;

void swap(int* a, int* b) { int t = *a; *a = *b; *b = t; }

void heapifyUp(int i) {
    int parent = (i - 1) / 2;
    if (i > 0 && heap[i] < heap[parent]) {
        swap(&heap[i], &heap[parent]);
        heapifyUp(parent);
    }
}

void heapifyDown(int i) {
    int smallest = i, left = 2*i+1, right = 2*i+2;
    if (left  < size && heap[left]  < heap[smallest]) smallest = left;
    if (right < size && heap[right] < heap[smallest]) smallest = right;
    if (smallest != i) {
        swap(&heap[i], &heap[smallest]);
        heapifyDown(smallest);
    }
}

void insert(int val) {
    if (size >= MAX) { printf("Heap full!\n"); return; }
    heap[size++] = val;
    heapifyUp(size - 1);
    printf("Inserted %d\n", val);
}

void deleteRoot() {
    if (size == 0) { printf("Heap is empty!\n"); return; }
    printf("Deleted root: %d\n", heap[0]);
    heap[0] = heap[--size];
    heapifyDown(0);
}

void display() {
    if (size == 0) { printf("Heap is empty!\n"); return; }
    printf("Heap: ");
    for (int i = 0; i < size; i++) printf("%d ", heap[i]);
    printf("\n");
}
```

```c
int main() {
    int choice, val;
    while (1) {
        printf("\n--- Min Heap Menu ---\n");
        printf("1.Insert  2.Delete Root  3.Display  4.Exit\n");
        printf("Enter choice: "); scanf("%d", &choice);

        switch (choice) {
            case 1: printf("Enter value: "); scanf("%d", &val); insert(val);
                break;
            case 2: deleteRoot(); break;
            case 3: display(); break;
            case 4: return 0;
            default: printf("Invalid choice!\n");
        }
    }
}
```

OUTPUT :

```
--- Min Heap Menu ---
1.Insert  2.Delete Root  3.Display  4.Exit
Enter choice: 1
Enter value: 1 11 31
Inserted 1

--- Min Heap Menu ---
1.Insert  2.Delete Root  3.Display  4.Exit
Enter choice: Invalid choice!

--- Min Heap Menu ---
1.Insert  2.Delete Root  3.Display  4.Exit
Enter choice: Invalid choice!

--- Min Heap Menu ---
1.Insert  2.Delete Root  3.Display  4.Exit
Enter choice: 2
Deleted root: 1

--- Min Heap Menu ---
1.Insert  2.Delete Root  3.Display  4.Exit
Enter choice: 3
Heap is empty!

--- Min Heap Menu ---
1.Insert  2.Delete Root  3.Display  4.Exit
Enter choice:
```

3. Write a C program to implement a **Priority Queue** using a **Binary Heap** (Min Heap or Max Heap). The program should support the following operations:
   (i) **Insert (Enqueue with Priority):** Insert an element along with its priority into the queue. Maintain heap property using heapify-up.
   (ii) **Delete (Dequeue Highest Priority Element):** Remove the element with the highest priority: (Min Heap → smallest element) and (Max Heap → largest element). Maintain heap property using heapify-down.
   (iii) **Peek:** Display the element with the highest priority without removing it.
   (iv) **Display:** Print all elements of the priority queue.

```c
#include <stdio.h>
#define MAX 100

struct Item {
    int value, priority;
};

struct Item pq[MAX];
int size = 0;

void swap(int i, int j) {
    struct Item t = pq[i]; pq[i] = pq[j]; pq[j] = t;
}

void heapifyUp(int i) {
    int p = (i - 1) / 2;
    if (i > 0 && pq[i].priority < pq[p].priority) {
        swap(i, p);
        heapifyUp(p);
    }
}

void heapifyDown(int i) {
    int smallest = i, l = 2*i+1, r = 2*i+2;
    if (l < size && pq[l].priority < pq[smallest].priority) smallest = l;
    if (r < size && pq[r].priority < pq[smallest].priority) smallest = r;
    if (smallest != i) { swap(i, smallest); heapifyDown(smallest); }
}

void enqueue(int val, int pri) {
    if (size >= MAX) { printf("Queue full!\n"); return; }
    pq[size].value = val;
    pq[size].priority = pri;
    heapifyUp(size++);
    printf("Inserted value=%d, priority=%d\n", val, pri);
}

void dequeue() {
    if (size == 0) { printf("Queue is empty!\n"); return; }
    printf("Dequeued: value=%d, priority=%d\n", pq[0].value, pq[0].priority);
    pq[0] = pq[--size];
    heapifyDown(0);
}

void peek() {
    if (size == 0) { printf("Queue is empty!\n"); return; }
    printf("Highest priority: value=%d, priority=%d\n", pq[0].value, pq[0].priority);
}
```

```c
void display() {
    if (size == 0) { printf("Queue is empty!\n"); return; }
    printf("Priority Queue (value:priority):\n");
    for (int i = 0; i < size; i++)
        printf("  [%d:%d]\n", pq[i].value, pq[i].priority);
}

int main() {
    int choice, val, pri;
    while (1) {
        printf("\n--- Priority Queue Menu ---\n");
        printf("1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit\n");
        printf("Enter choice: "); scanf("%d", &choice);

        switch (choice) {
            case 1: printf("Enter value and priority: "); scanf("%d %d", &val, &pri);
                    enqueue(val, pri); break;
            case 2: dequeue(); break;
            case 3: peek(); break;
            case 4: display(); break;
            case 5: return 0;
            default: printf("Invalid choice!\n");
        }
    }
}
```

OUTPUT :

```
--- Priority Queue Menu ---
1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter value and priority: 12 1
Inserted value=12, priority=1

--- Priority Queue Menu ---
1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 3
Highest priority: value=12, priority=1

--- Priority Queue Menu ---
1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice:
```